
UNIT 10 CLASSIFICATION

- 10.0 Introduction
 - 10.1 Objectives
 - 10.2 Classification – An Overview
 - 10.3 k-NN Algorithm
 - 10.4 Decision Tree Classifier
 - 10.5 Bayesian Classification
 - 10.6 Support Vector Machines
 - 10.7 Rule Based Classification
 - 10.8 Model Evaluation and Selection
 - 10.9 Summary
 - 10.10 Solutions/Answers
 - 10.11 Further Readings
-

10.0 INTRODUCTION

In the earlier unit, we have studied Mining frequent patterns and associations covering the topics market basket analysis, classification of frequent pattern mining, association rule mining, Apriori algorithm, mining multilevel association rules etc.. In this unit, we will focus on an important topic of Data mining called Classification.

Knowledge discovery from datasets is a part of data mining. Data mining tools and methods are applied to extract patterns and features from large amount of data, which can then be applied to other datasets. The classification technique, which can handle a larger range of data, is gaining prominence. It is one of the most commonly used technique's when it comes to classifying large sets of data. This method of data analysis includes algorithms adapted to the data quality. The algorithm that performs the classification is the classifier while the observations are the instances.

For example, companies use this approach to learn about the behavior and preferences of their customers. With classification, you can distinguish between data that is useful to your goal and data that is not relevant. Another example of this would be your own email service, which can identify spam and important messages.

In this unit you are going to study the concept of classification in data mining and various classifiers.

10.1 OBJECTIVES

After completing this unit, you will be able to

- define Classification in Data Mining;
- understand the approach to data classification;
- list various types of Classification Techniques;
- understand and demonstrate various Classification Techniques;
- discuss the advantages, disadvantages and applications of various Classification Techniques, and

- discuss various approaches for Model Evaluation and Model Selection.

10.2 CLASSIFICATION – AN OVERVIEW

Classification is a data mining process that assigns items in a collection to target categories or classes. The objective of classification is to accurately predict the target class for each record in the data. For example, a classification model used to identify loan applicants as low, medium, or high credit risks. A classification model that predicts credit risk could be developed based on observed data for many loan applicants over a period of time. In addition to the historical credit rating, the data might track employment history, home ownership years of residence, number and type of investments, And so on. Credit rating would be the target, the other attributes are the predictors, and the data for each Customer constitute a case. Classifications are discrete and do not imply order. A predictive classifier with a numerical target uses a regression algorithm, not a classification algorithm. The simplest type of classification problem is binary classification. Where, the target attribute has only two values- *YES* or *NO*. Multiclass targets have more than two values: for example, low, medium, high, or unknown credit rating.

For example, spam detection in email service providers can be identified as a classification problem. This is a binary classification since there are only 2 classes as spam and not spam. A classifier utilizes some training data to understand how given input variables relate to the class. In this case, known *spam* and *non-spam* emails have to be used as the training data. When the classifier is trained accurately, it can be used to detect an unknown email.

10.2.1 General Approach to Classification

Data classification is a two-step process, consisting of:

- (i) a learning step (where a classification model is constructed) and
- (ii) a classification step (where the model is used to predict class labels for given data).

In the *first step*, a classifier is built describing a predetermined set of data classes or concepts. This is the learning step (or training phase), where a classification algorithm builds the classifier by analyzing or “learning from” a training set made up of database tuples and their associated class labels. A tuple, X , is represented by an n -dimensional attribute vector, $X = (x_1, x_2, \dots, x_n)$, depicting n measurements made on the tuple from n database attributes, respectively $A_1, A_2, \dots, A_n$. Each tuple, X , is assumed to belong to a predefined class as determined by another database attribute called the class label attribute. The class label attribute is discrete-valued and unordered. It is categorical (or nominal) in that each value serves as a category or class. The individual tuples making up the training set are referred to as training tuples. Because the class label of each training tuple is provided, this step is also known as supervised learning (i.e., the learning of the classifier is “supervised” in that it is told to which class each training tuple belongs). It contrasts with unsupervised learning (or clustering), in which the class label of each training tuple is not known, and the number or set of classes to be learned may not be known in advance.

In the *second step*, the model is used for classification. First, the predictive accuracy of the classifier is estimated. If we were to use the training set to measure the

classifier's accuracy, this estimate would likely be optimistic, because the classifier tends to overfit the data (i.e., during learning it may incorporate some particular anomalies of the training data that are not present in the general data set overall). Therefore, a test set is used, made up of test tuples and their associated class labels. They are independent of the training tuples, meaning that they were not used to construct the classifier. The accuracy of a classifier on a given test set is the percentage of test set tuples that are correctly classified by the classifier.

10.2.2 Applications of Classification Models

The classification in Data Mining has many applications in day-to-day life. A few Classification Applications in Data Mining are:

- Product Cart Analysis on the eCommerce platform uses the classification technique to associate the items into groups and create combinations of products to recommend. This is a very common Classification Applications in Data Mining
- The weather patterns can be predicted and classified based on parameters such as temperature, humidity, wind direction, and many more. These Classification Applications of Data Mining are used in daily life.
- The public health sector classifies the diseases based on the parameters like spread rate, severity, and a lot more. This helps in charting out strategies to mitigate diseases. These Classification Applications of Data Mining help in finding cures.
- Financial institutes use classification to determine the defaulters and help in determining the loan seekers, and other categories. These Classification Applications in Data Mining helps in finding the target audience much easier.
- Spam detection e-mails based on the header and content of the document.
- Classification of students according to their qualifications.
- Patients are classified according to their medical history.
- Classification can be used for the approval of credit.
- Facial key points detection.
- Drugs classification.
- Pedestrian's detection in an automotive car driving.
- Cancer tumor cells identification.
- Sentiment Analysis.

Let us see the definitions for some of the terminologies those will encounter in classification:

- *Classifier*: An algorithm that maps the input data to a specific category.
- *Classification Model*: A classification model tries to draw some conclusion from the input values given for training. It will predict the class labels/categories for the new data.
- *Feature*: A feature is an individual measurable property of a phenomenon

being observed.

- *Binary Classification:* Classification task with two possible outcomes. *Example:* Gender classification (Male / Female).
- *Multi-class classification:* Classification with more than two classes. In multi class classification each sample is assigned to one and only one target label. *Example:* An animal can be cat or dog but not both at the same time.
- *Multi-label Classification:* Classification task where each sample is mapped to a set of target labels (more than one class).

Example: A news article can be about sports, a person, and location at the same time.

10.23 Classification Algorithms in Data Mining

Classification is the operation of separating various entities into several classes. These classes can be defined by business rules, class boundaries, or some mathematical function. The classification operation may be based on a relationship between a known class assignment and characteristics of the entity to be classified. This type of classification is called supervised. If no known examples of a class are available, the classification is unsupervised. The most common unsupervised classification approach is clustering, which we will be studying in the next Unit.

Classification algorithm finds relationships between the values of the predictors and the values of the target. Different Classification algorithms use different techniques for finding relationships. These relationships are summarized in a model, which can then be applied to a different dataset in which the class assignments are unknown. Classification models are tested by comparing the predicted values to known target values in a set of test data. Classification belongs to the category of supervised learning where the targets also provided with the input data.

Classification is a highly popular aspect of data mining. As a result, data mining has many classifiers/classification algorithms:

- Logistic regression
- Linear regression
- K-Nearest Neighbours Algorithm (kNN)
- Decision trees
- Rule-based Classification
- Bayesian Classification
- Random Forest
- Naive Bayes
- Support Vector Machines

We will study the details of some to the popular Classifiers in the following sections. To start with, we will study k-NN classification technique in the following section.

10.3 k-NN ALGORITHM

The k-Nearest Neighbors (k-NN) algorithm is a data classification method for estimating the likelihood that a data point will become a member of one group or another based on what group the data points nearest to it belong to.

The k-nearest neighbor algorithm is a type of supervised machine learning algorithm used to solve classification and regression problems. However, it's mainly used for classification problems. k-NN is a lazy learning and non-parametric algorithm.

It's called a lazy learning algorithm or lazy learner because it doesn't perform any training when you supply the training data. Instead, it just stores the data during the training time and doesn't perform any calculations. It doesn't build a model until a query is performed on the dataset.

It's considered a non-parametric method because it doesn't make any assumptions about the underlying data distribution. Simply put, k-NN tries to determine what group a data point belongs to by looking at the data points around it.

Consider there are two groups, A and B. To determine whether a data point is in group A or group B, the algorithm looks at the states of the data points near it. If the majority of data points are in group A, it's very likely that the data point in question is in group A and vice versa.

In short, k-NN involves classifying a data point by looking at the nearest annotated data point, also known as the nearest neighbor.

Don't confuse k-NN classification with K-means clustering. K-NN is a supervised classification algorithm that classifies new data points based on the nearest data points. On the other hand, K-means clustering is an unsupervised clustering algorithm that groups data into a K number of clusters which you will be learning in the next Unit.

10.3.1 Working of k-NN

As mentioned above, the k-NN algorithm is predominantly used as a classifier. Let's take a look at how kNN works to classify unseen input data points.

k-NN classification is easy to understand and simple to implement. It's ideal in situations where the data points are well defined or non-linear. In essence, k-NN performs a voting mechanism to determine the class of an unseen observation. This means that the class with the majority vote will become the class of the data point in question. If the value of K is equal to one, then we'll use only the nearest neighbor to determine the class of a data point. If the value of K is equal to ten, then we'll use the ten nearest neighbors, and so on.

To put that into perspective, consider an unclassified data point X. There are several data points with known categories, A and B, in a scatter plot. Suppose the data point X is placed near group A. As you know, we classify a data point by looking at the nearest annotated points. If the value of K is equal to one, then we'll use only one nearest neighbor to determine the group of the data point. In this case, the data point X belongs to group A as its nearest neighbor is in the same group. If group A has more than ten data points and the value of K is equal to 10, then the data point X will still belong to group A as all its nearest neighbors are in the same group.

Suppose another unclassified data point Y is placed between group A and group B. If K is equal to 10, we pick the group that gets the most votes, meaning that we classify Y to the group in which it has the most number of neighbors. For example, if Y has seven neighbors in group B and three neighbors in group A, it belongs to group B. The fact that the classifier assigns the category with the highest number of votes is true regardless of the number of categories present.

You might be wondering how the distance metric is calculated to determine whether a data point is a neighbor or not. There are four ways to calculate the distance measure between the data point and its nearest neighbor: Euclidean distance, Manhattan distance, Hamming distance, and Minkowski distance. Out of the three, Euclidean distance is the most commonly used distance function or metric.

10.3.2 K-Nearest Neighbors Algorithm

The following is the pseudocode for KNN:

1. Load the data
2. Choose K value
3. For each data point in the data:
 - o Find the Euclidean distance to all training data samples
 - o Store the distances on an ordered list and sort it
 - o Choose the top K entries from the sorted list
 - o Label the test point based on the majority of classes present in the selected points
4. End

To validate the accuracy of the k-NN classification, a confusion matrix is used. Other statistical methods such as the likelihood-ratio test are also used for validation.

Here are some of the areas where the k-Nearest Neighbor algorithm can be used:

- **Credit rating:** The k-NN algorithm helps determine an individual's credit rating by comparing them with the ones with similar characteristics.
- **Loan approval:** Similar to credit rating, the k-nearest neighbor algorithm is beneficial in identifying individuals who are more likely to default on loans by comparing their traits with similar individuals.
- **Data Preprocessing:** Datasets can have many missing values. The k-NN algorithm is used for a process called missing data imputation that estimates the missing values.
- **Recognition:** The ability of the k-NN algorithm to identify patterns creates a wide range of applications. For example, it helps detect patterns in credit card usage and spot unusual patterns. Pattern detection is also useful in identifying patterns in customer purchase behavior.
- **Stock Price Prediction:** Since the k-NN algorithm has a flair for predicting the values of unknown entities, it's useful in predicting the future value of stocks based on historical data.
- **Recommendation Systems:** Since k-NN can help find users of similar characteristics, it can be used in recommendation systems. For example, it can be used in an online video streaming platform to suggest content a user is more likely to watch by analyzing what similar users watch.

- **Computer Vision:** The k-NN algorithm is used for image classification. Since it's capable of grouping similar data points, for example, grouping cats together and dogs in a different class, it's useful in several computer vision applications.

10.3.3 Advantages and Disadvantages of KNN

Some of the advantages of using the k-Nearest Neighbors algorithm are:

- It's easy to understand and simple to implement
- It can be used for both classification and regression problems
- It's ideal for non-linear data since there's no assumption about underlying data
- It can naturally handle multi-class cases
- It can perform well with enough representative data.

The disadvantages of using the k-Nearest Neighbors algorithm:

- Associated computation cost is high as it stores all the training data
- Requires high memory storage
- Need to determine the value of K
- Prediction is slow if the value of N is high
- Sensitive to irrelevant features

We will study Decision Tree classification technique in the next section.

10.4 DECISION TREE CLASSIFIER

Decision tree classifier is the most effective and common prediction and classification method. It is a simple and widely used classification technique. It applies a straight forward idea to solve the classification problem. Decision Tree Classifier poses a series of carefully crafted questions about the attributes of the test record. Each time it receives an answer, a follow-up question is asked until a conclusion about the class label of the record is reached.

A decision tree is a flow chart like tree structure, where each internal node denotes a test on an attribute, each branch represents an outcome of the test, and leaf nodes represent classes or class distributions. The top most node in a tree is the root node. Normally, internal nodes are denoted by rectangles and leaf nodes are denoted by ovals. A typical decision tree for playing tennis is shown in Figure 1 given below:

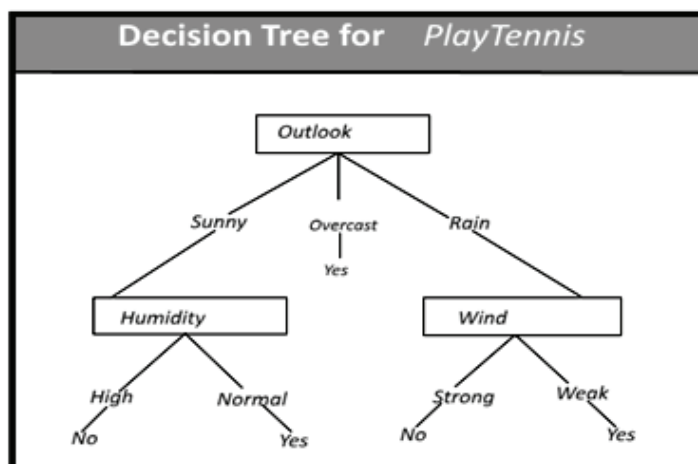


Figure 1: Decision Tree for Playing Tennis

In order to classify an unknown sample, the attribute values of the sample are tested against the decision tree. A path is traced from the root to a leaf node that holds the class prediction for that sample. Decision trees can easily be converted to classification rules. A basic decision tree algorithm is summarized in the next section.

10.4.1 Basic Algorithm for Inducing a Decision Tree from Training Samples

Below given is the basic decision tree algorithm for learning decision trees.

Algorithm: `Generate_decision_tree`. Generate a decision tree from the given Training Data

Input: The training samples, *samples*, represented by discrete-valued attributes; the set of candidate attributes, *attribute-list*.

Output: A decision tree.

- (1) create a node *N*;
- (2) If *samples* are all of the same class, *C* then
- (3) Return *N* as a leaf node labeled with the class *C*;
- (4) If *attribute-list* is empty then
- (5) return *N* as a leaf node labeled with the most common class in *samples*;
// majority voting
- (6) select *test-attribute*, the attribute among *attribute-list* with the highest information gain;
- (7) label node *N* with *test-attribute*;
- (8) for each known value a_i of *test-attribute* //partition the samples
- (9) grow a branch from node *N* for the condition *test-attribute* = a_i ;
- (10) let s_i be the set of samples in *samples* for which *test-attribute*
= a_i ; //a partition
- (11) If s_i is empty then
- (12) attach a leaf labeled with the most common class in *samples*;
- (13) else attach the node returned by `Generate_decision_tree( $s_i$ , attribute-list - test-attribute)`;

The algorithm summarized above is a version of ID3, a well known decision tree induction algorithm. The basic strategy is as follows:

- The tree starts as a single node representing the training samples (step 1).
- If the samples are all of the same class, then the node becomes a leaf and is labeled with that class (steps 2 and 3).
- Otherwise, the algorithm uses an entropy based measure known as information gain as a heuristic for selecting the attribute that will best separate the samples into individual classes (step 6). This attribute becomes the “test” or “decision” attribute at the node (step 7). In this version of the

algorithm, all attributes are categorical, that is, discrete-valued. Continuous-valued attributes must be discretized.

- A branch is created for each known value of the test attributes, and the samples are partitioned accordingly (steps 8-10).
- The algorithm uses the same process recursively to form a decision tree for the samples at each partition. Once an attribute has occurred at a node, it need not be considered in any of the node's descendents (step 13).

The recursive partitioning stops only when any one of the following conditions is TRUE:

- All samples for a given node belong to the same class (steps 2 and 3), or
- There are no remaining attributes on which the samples may be further partitioned (step 4). In this case, majority voting is employed (step 5). This involves converting the given node into leaf and labeling it with the class in majority among samples. Alternatively, the class distribution of the node samples may be stored.
- There are no samples for the branch test-attribute = a_i (step 11). In this case, a leaf is created with the majority class in samples (step 12).

10.4.2 Attribute Selection Measure

An attribute selection measure is a heuristic for selecting the splitting criterion that “best” separates a given data partition, D , of class-labeled training tuples into individual classes. If we were to split D into smaller partitions according to the outcomes of the splitting criterion, ideally each partition would be pure (i.e., all the tuples that fall into a given partition would belong to the same class). The attribute selection measure provides a ranking for each attribute describing the given training tuples. The attribute having the best score for the measure is chosen as the splitting attribute for the given tuples. If the splitting attribute is continuous-valued or if we are restricted to binary trees, then, respectively, either a split point or a splitting subset must also be determined as part of the splitting criterion.

The three popular attribute selection measures are:

- Information gain
- Gain ratio
- Gini index

In fact, these three are closely related to each other. *Information Gain*, which is also known as Mutual information, is devised from the transition of Entropy, which in turn comes from Information Theory. *Gain Ratio* is a complement of Information Gain, was born to deal with its predecessor's major problem. *Gini Index*, on the other hand, was developed independently with its initial intention is to assess the income dispersion of the countries but then be adapted to work as a heuristic for splitting optimization.

10.4.2.1 Entropy

Entropy or Information Entropy, or just, is a measurement of the uncertainty in data. Entropy measures the diversification of the labels.

- Low Entropy indicates that the data labels are quite uniform.

Example: Suppose a dataset has 100 samples. Among those, there are 1 Positive and 99 Negative labeled data points. In this case, the Entropy is very low. In an extreme case, suppose all the 100 samples are Positive, then the Entropy is at its minimum or zero.

- High Entropy means the labels are in chaos.

Example: A dataset with 45 Positive samples and 55 Negative samples has a very high Entropy. The extreme case, when the Entropy is highest happens when exactly half of the data belongs to each of the labels.

In another point of view, the Entropy measures how hard we guess the label of a randomly taken sample from the dataset. If most of the data have the same label, says, Positive, meaning the Entropy is low, thus we can bet the label of the random sample is Positive with confidence. On the flip side, if the Entropy is high, meaning the probabilities of the sample to fall into the 2 classes are comparable, making us hard to make a guess.

The formula of Entropy is given by:

$$H(X) = - \sum_{i=1}^n p(x_i) \log P(x_i)$$

where,

X = random variable or process

X_i = possible outcomes

$p(X_i)$ = probability of possible outcomes.

Let's have a dataset made up of three colors - red, purple, and yellow.

If we have one red, three purple, and four yellow observations in our set, our equation becomes:

$$E = -(p_r \log_2 p_r + p_p \log_2 p_p + p_y \log_2 p_y)$$

where p_r , p_p and p_y are the probabilities of choosing a red, purple and yellow example respectively. We have $p_r=1/8$ because only 1/8 of the dataset represents red. 3/8 of the dataset is purple hence $p_p=3/8$. Finally, $p_y=4/8$ since half the dataset is yellow. As such, we can represent p_y as $p_y=1/2$.

Our equation now becomes:

$$E = -(1/8 \log_2 (1/8) + 3/8 \log_2 (3/8) + 4/8 \log_2 (4/8))$$

Our entropy would be: 1.411.41

You might wonder, what happens when all observations belong to the same class? In such a case, the entropy will always be zero.

$$\begin{aligned} E &= -(1 \log_2 1) \\ &= 0 \end{aligned}$$

Such a dataset has no impurity. This implies that such a dataset would not be useful for learning. However, if we have a dataset with say, two classes, half made up of yellow and the other half being purple, the entropy will be one.

$$E = -((0.5 \log_2 0.5) + (0.5 \log_2 0.5))$$

$$= 1$$

This kind of dataset is good for learning.

10.4.2.2 Information Gain

The concept of entropy plays an important role in measuring the information gain. Information gain is based on the information theory. Information gain is used for determining the best features/attributes that render maximum information about a class. It follows the concept of entropy while aiming at decreasing the level of entropy, beginning from the root node to the leaf nodes.

Information gain computes the difference between entropy before and after split and specifies the impurity in class elements.

It can help us determine the quality of splitting, as we shall soon see. The calculation of information gain should help us understand this concept better.

$$\text{Gain} = E_{\text{parent}} - E_{\text{children}}$$

The term Gain represents information gain. E_{parent} is the entropy of the parent node and $E_{\{\text{children}\}}$ is the average entropy of the child nodes. Let's use an example to visualize information gain and its calculation.

Suppose we have a dataset with two classes. This dataset has 5 purple and 5 yellow examples. The initial value of entropy will be given by the equation below. Since the dataset is balanced, we expect the answer to be 1.

$$E_{\text{initial}} = -((0.5 \log_2 0.5) + (0.5 \log_2 0.5))$$

$$= 1$$

Say we split the dataset into two branches. One branch ends up having four values while the other has six. The left branch has four purples while the right one has five yellows and one purple.

We mentioned that when all the observations belong to the same class, the entropy is zero since the dataset is pure. As such, the entropy of the left branch $E_{\text{left}} = 0$. On the other hand, the right branch has five yellows and one purple.

Thus:

$$E_{\text{right}} = -\left(\frac{5}{6} \log_2 \left(\frac{5}{6}\right) + \frac{1}{6} \log_2 \left(\frac{1}{6}\right)\right)$$

A perfect split would have five examples on each branch. This is clearly not a perfect split, but we can determine how good the split is. We know the entropy of each of the two branches. We weight the entropy of each branch by the number of elements each contains.

This helps us calculate the quality of the split. The one on the left has 4, while the other has 6 out of a total of 10. Therefore, the weighting goes as shown below:

$$E_{\text{split}} = 0.6 * 0.65 + 0.4 * 0$$

$$= 0.39$$

The entropy before the split, which we referred to as initial entropy $E_{\text{initial}} = 1$. After splitting, the current value is 0.39. We can now get our information gain,

which is the entropy we “lost” after splitting.

$$\text{Gain} = 1 - 0.39$$

$$= 0.61$$

The more the entropy removed, the greater the information gain. The higher the information gain, the better the split.

10.4.2.3 Gain Ratio

Gain Ratio was proposed by John Ross Quinlan. Gain Ratio or Uncertainty Coefficient is used to normalize the information gain of an attribute against how much entropy that attribute has. Formula of Gain Ratio is given by:

$$\text{Gain Ratio} = \text{Information Gain} / \text{Entropy}$$

From the above formula, it can be stated that if entropy is very small, then the gain ratio will be high and vice versa.

Be selected as splitting criterion, Quinlan proposed following procedure:

1. First, determine the information gain of all the attributes, and then compute the average information gain.
2. Second, calculate the gain ratio of all the attributes whose calculated information gain is larger or equal to the computed average information gain, and then pick the attribute of higher gain ratio to split.

10.4.2.4 Gini Index

The Gini index or Gini coefficient or Gini impurity computes the degree of probability of a specific variable that is wrongly being classified when chosen randomly and a variation of Gini coefficient. It works on categorical variables, provides outcomes both be “successful” or “failure”, and hence, conducts binary splitting only.

The degree of Gini index varies from 0 to 1,

- Where 0 depicts that all the elements be allied to a certain class, or only one class exists there.
- The gini index of value as 1 signifies that all the elements are randomly distributed across various classes, and
- A value of 0.5 denotes the elements are uniformly distributed into some classes.

It was proposed by Leo Breiman in 1984 as an impurity measure for decision tree learning. The Gini Index is determined by deducting the sum of squared of probabilities of each class from one, mathematically, Gini Index can be expressed as:

$$\text{Gini Index} = 1 - \sum_{i=1}^n (P_i)^2$$

where P_i denotes the probability of an element being classified for a distinct class.

10.4.2.5 Gini Index vs Information Gain

Take a look below for the getting discrepancy between Gini Index and Information

Gain,

- The Gini Index facilitates the bigger distributions so easy to implement whereas the Information Gain favors lesser distributions having small count with multiple specific values.
- The method of the Gini Index is used by CART algorithms, in contrast to it, Information Gain is used in ID3, C4.5 algorithms.
- Gini index operates on the categorical target variables in terms of “success” or “failure” and performs only binary split, in opposite to that Information Gain computes the difference between entropy before and after the split and indicates the impurity in classes of elements.

10.4.3 Tree Pruning

When a decision tree is built, many of the branches will reflect anomalies in the training data due to noise or outliers. Tree pruning methods address this problem of *overfitting* the data. Such methods typically use statistical measures to remove the least reliable branches, generally resulting in faster classification and an improvement in the ability of the tree to correctly classify independent test data.

10.4.3.1 How does Tree Pruning works?

There are two common approaches to tree pruning: *prepruning* and *postpruning*. In the prepruning approach, a tree is “pruned” by halting its construction early (example, by deciding not to further split or partition the subset of training tuples at a given node). The second and more common approach is postpruning, which removes subtrees from a “fully grown” tree. A subtree at a given node is pruned by removing its branches and replacing it with a leaf.

Alternatively, prepruning and postpruning may be interleaved for a combined approach. Postpruning requires more computation than prepruning, yet generally leads to a more reliable tree. No single pruning method has been found to be superior over all others.

Decision trees can suffer from repetition and replication, making them overwhelming to interpret. Repetition occurs when an attribute is repeatedly tested along a given branch of the tree. In Replication, duplicate subtrees exist within the tree

10.4.4 Advantages and Limitations of Decision Tree Approaches

The advantages of decision tree approaches are:

- Decision trees are simple to understand and interpret.
- They require little data and are able to handle both numerical and categorical data.
- Decision trees can produce comprehensible rules.
- Classification of decision trees without much computation.
- Decision trees can accommodate both categorical and continuous variables.
- Decision trees clearly show which fields for prediction or classification are most important.
- They are strong in nature, therefore, they perform well even if its

assumptions are somewhat violated by the true model from which the data were generated.

- Decision trees perform well with large data in a short time.
- Nonlinear relationships between parameters do not affect tree performance.
- The best feature of using trees for analytics - easy to interpret and explain to executives.

The limitations of decision tree approaches are:

- Decision trees are less suited to estimate tasks where the goal is to predict a constant attribute value.
- The decision trees are vulnerable to mistakes in many class problems and relatively limited numbers of training instances.
- Decision tree can be costly to train computationally. The decision-making method is computationally costly. Each splitting field must be sorted at each node before the best split can be identified. Combinations of fields are used in some algorithms and search must be made for optimal combined weights. Pruning algorithms can also be costly as many sub-trees of candidates have to be created and compared.
- **Data Fragmentation:** Each split in a tree leads to a reduced dataset under consideration. And, hence the model created at the split will potentially introduce bias.
- **High Variance and Unstable:** As a result of the greedy strategy applied by decision tree's variance in finding the right starting point of the tree can greatly impact the final result. i.e. small changes early on can have big impacts later. So- if for example you draw two different samples from your universe, the starting points for both the samples could be very different (and may even be different variables) this can lead to totally different results.

We will study Bayesian classification technique which includes Baye's theorem, Naïve Bayes etc.. in the next section.

10.5 BAYESIAN CLASSIFICATION

Bayesian classifiers are statistical classifiers. They can predict class membership probabilities, such as the probability that a given tuple belongs to a particular class.

Bayesian classification is based on Bayes' theorem, described below. Studies comparing classification algorithms have found a simple Bayesian classifier known as the Naïve Bayesian classifier to be comparable in performance with decision tree and selected neural network classifiers. Bayesian classifiers exhibit high accuracy and speed when applied to large databases. Naïve Bayesian classifiers assume that the effect of an attribute value on a given class is independent of the values of the other attributes. This assumption is called class conditional independence. It is made to simplify the computations involved and, in this sense, is considered "Naïve."

10.5.1 Bayes' Theorem

Bayes' theorem is named after Thomas Bayes, English clergyman who did early

work in probability and decision theory during the 18th century.

Let X is a data tuple. In Bayesian terms, X is considered “evidence.” As usual, it is described by measurements made on a set of n attributes.

Let H be some hypothesis, such as that the data tuple X belongs to a specified class C . For classification problems, we want to determine $P(H|X)$, the probability that the hypothesis H holds given the “evidence” or observed data tuple X . In other words, we are looking for the probability that tuple X belongs to class C , given that we know the attribute description of X . $P(H|X)$ is the posterior probability, or a posteriori probability, of H conditioned on X .

For example, suppose our world of data tuples is confined to customers described by the attributes age and income, respectively, and that X is a 35-year-old customer with an income of Rs.4,00,000. Suppose that H is the hypothesis that our customer will buy a computer. Then $P(H|X)$ reflects the probability that customer X will buy a computer given that we know the customer’s age and income.

In contrast, $P(H)$ is the prior probability, or a priori probability, of H . For our example, this is the probability that any given customer will buy a computer, regardless of age, income, or any other information, for that matter. The posterior probability, $P(H|X)$, is based on more information (e.g., customer information) than the prior probability, $P(H)$, which is independent of X .

Similarly, $P(X|H)$ is the posterior probability of X conditioned on H . That is, it is the probability that customers, X , is 35 years old and earns Rs. 4,00,000, given that we know the customer will buy a computer.

$P(X)$ is the prior probability of X . Using our example, it is the probability that a person from our set of customers is 35 years old and earns Rs.4,00,000.

How are these probabilities estimated? $P(H)$, $P(X|H)$, and $P(X)$ may be estimated from the given data, as we shall see below. Bayes’ theorem is useful in that it provides a way of calculating the posterior probability, $P(H|X)$, from $P(H)$, $P(X|H)$, and $P(X)$.

Bayes’ theorem is

$$P(H|X) = \frac{P(X|H)P(H)}{P(X)}$$

In the next section, we will look at how Bayes’ theorem is used in the Naive Bayesian classification.

10.5.2 Naive- Bayesian Classification

The Naïve Bayesian classifier, or simple Bayesian classifier, works as follows:

- Let D be a training set of tuples and their associated class labels.
- Each tuple is represented by an n -dimensional attribute vector, $X = (x_1, x_2, \dots, x_n)$, depicting n measurements made on the tuple from n attributes, respectively, $A_1, A_2, \dots, A_n$.
- Suppose that there are m classes, $C_1, C_2, \dots, C_m$. Given a tuple, X , the classifier will predict that X belongs to the class having the highest posterior probability, conditioned on X . That is, the naïve Bayesian classifier predicts that tuple X belongs to the class C_i if and only if

$$P(C_i|X) > P(C_j|X) \text{ for } 1 \leq j \leq m; j \neq i$$

Thus we maximize $P(C_i|X)$. The class C_i for which $P(C_i|X)$ is maximized is called the maximum posteriori hypothesis. Using Bayes' theorem,

$$P(C_i|X) = \frac{P(X|C_i)P(C_i)}{P(X)}$$

- As $P(X)$ is constant for all classes, only $P(X|C_i)P(C_i)$ need be maximized. If the class prior probabilities are not known, then it is commonly assumed that the classes are equally likely, that is, $P(C_1) = P(C_2) = \dots = P(C_m)$, and we would therefore maximize $P(X|C_i)$. Otherwise, we maximize $P(X|C_i)P(C_i)$. Note that the class prior probabilities may be estimated by $P(C_i) = |C_i, D|/|D|$, where $|C_i, D|$ is the number of training tuples of class C_i in D .
- Given data sets with many attributes, it would be extremely computationally expensive to compute $P(X|C_i)$. In order to reduce computation in evaluating $P(X|C_i)$, the naive assumption of class conditional independence is made. This presumes that the values of the attributes are conditionally independent of one another, given the class label of the tuple (i.e., that there are no dependence relationships among the attributes). Thus,

$$\begin{aligned} P(X|C_i) &= \prod_{k=1}^n P(x_k|C_i) \\ &= P(x_1|C_i) \times P(x_2|C_i) \times \dots \times P(x_n|C_i) \end{aligned}$$

We can easily estimate the probabilities $P(x_1|C_i)$, $P(x_2|C_i)$, $\dots$, $P(x_n|C_i)$ from the training tuples. Recall that here x_k refers to the value of attribute A_k for tuple X . For each attribute, we look at whether the attribute is categorical or continuous-valued.

For instance, to compute $P(X|C_i)$, we consider the following:

- If A_k is categorical, then $P(x_k|C_i)$ is the number of tuples of class C_i in D having the value x_k for A_k , divided by $|C_i, D|$, the number of tuples of class C_i in D .
- If A_k is continuous-valued, then we need to do a bit more work, but the calculation is pretty straightforward. A continuous-valued attribute is typically assumed to have a Gaussian distribution with a mean μ and standard deviation σ , defined by

$$g(x, \mu, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}},$$

$$P(x_k|C_i) = g(x_k, \mu_{C_i}, \sigma_{C_i})$$

For example, let $X = (35, \$40,000)$, where A_1 and A_2 are the attributes age and income, respectively. Let the class label attribute be buys_computer. The associated class label for X is yes (i.e., buys_computer = yes). Let's suppose that age has not been discretized and therefore exists as a continuous-valued attribute. Suppose that from the training set, we find that customers in D who buy a computer are between 12 and 38 years of age. In other words, for attribute age and this class, we have

$\mu = 38$ years and $\sigma = 12$.

We can plug these quantities, along with $x_1 = 35$ for our tuple X , into equation to estimate $P(\text{age} = 35 | \text{buys_computer} = \text{yes})$.

To predict the class label of X , $P(X|C_i)P(C_i)$ is evaluated for each class C_i . The classifier predicts that the class label of tuple X is the class C_i if and only if

$$P(X|C_i)P(C_i) > P(X|C_j)P(C_j) \text{ for } 1 \leq j \leq m, j \neq i$$

In other words, the predicted class label is the class C_i for which $P(X|C_i)P(C_i)$ is the maximum.

Bayesian classifiers have the minimum error rate in comparison to all other classifiers. However, in practice this is not always the case, owing to inaccuracies in the assumptions made for its use, such as class-conditional independence, and the lack of available probability data.

Bayesian belief networks are graphical models, which unlike naïve Bayesian classifiers allow the representation of dependencies among subsets of attributes. Bayesian belief networks can also be used for classification.

10.5.3 Advantages and Disadvantage of Naïve Bayes classifier

Some to the **advantages** of the Naïve Bayes Classifier are:

- Naive Bayes Algorithm is a fast, highly scalable algorithm.
- Naive Bayes can be use for Binary and Multiclass classification. It provides different types of Naive Bayes Algorithms like GaussianNB, MultinomialNB, BernoulliNB.
- It is a simple algorithm that depends on doing a bunch of counts.
- Great choice for Text Classification problems. It's a popular choice for spam email classification.
- It can be easily train on small dataset

The **disadvantage** of Naïve Bayes Classifier is:

- Naïve Bayes can learn individual features importance but can't determine the relationship among features.

Common applications of Naïve Bayes algorithm are in Spam filtering. Gmail from Google uses Naïve Bayes algorithm for filtering spam emails. Sentiment analysis is another area where Naïve Bayes can calculate the probability of emotions expressed in the text being positive or negative. Leading web portals may understand the reaction of customers to their new products based on sentiment analysis.

We will focus on Support Vector Machines classification technique in the following section.

10.6 SUPPORT VECTOR MACHINES

Support vector machines (SVM) are a class of statistical models first developed in the mid-1960s by Vladimir Vapnik. In later years, the model has evolved considerably into one of the most flexible and effective machine learning tools available. It is a supervised learning algorithm which can be used to solve both classification and

regression problem, even though the current focus is on classification only.

To put it in a nutshell, this algorithm looks for a linearly separable hyperplane, or a decision boundary separating members of one class from the other. If such a hyperplane exists, the work is done! If such a hyperplane does not exist, SVM uses a nonlinear mapping to transform the training data into a higher dimension. Then it searches for the linear optimal separating hyperplane. With an appropriate nonlinear mapping to a sufficiently high dimension, data from two classes can always be separated by a hyperplane. The SVM algorithm finds this hyperplane using support vectors and margins. As a training algorithm, SVM may not be very fast compared to some other classification methods, but owing to its ability to model complex nonlinear boundaries, SVM has high accuracy. SVM is comparatively less prone to overfitting. SVM has successfully been applied to handwritten digit recognition, text classification, speaker identification etc..

10.6.1 When Data is Linearly Separable

Let us start with a simple two-class problem when data is clearly linearly separable as shown in the diagram below:

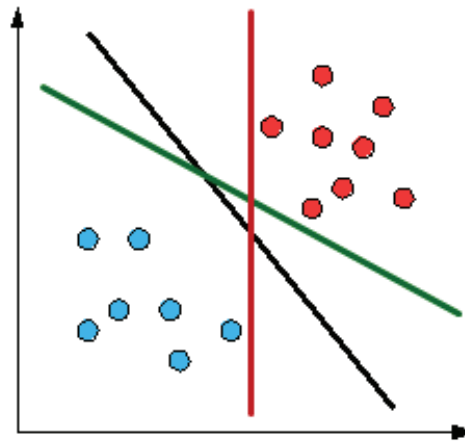


Figure 2: Linearly Separable

Let the i -th data point be represented by (X_i, Y_i) where X_i represents the feature vector and Y_i is the associated class label, taking two possible values $+1$ or -1 . In the Figure 2, above the balls having red color has class label $+1$ and the blue balls have class label -1 , say. A straight line can be drawn to separate all the members belonging to class $+1$ from all the members belonging to the class -1 . The two dimensional data above are clearly linearly separable.

In fact, an infinite number of straight lines can be drawn to separate the blue balls from the red balls.

The problem therefore is which among the infinite straight lines is optimal, in the sense that it is expected to have minimum classification error on a new observation. The straight line is based on the training sample and is expected to classify one or more test samples correctly.

As an illustration, if we consider the black, red and green lines in the Figure 2 above, is any one of them better than the other two? Or are all three of them equally well suited to classify? How is optimality defined here? Intuitively it is clear that if a line passes too close to any of the points, that line will be more sensitive to small

changes in one or more points. The green line is close to a red ball. The red line is close to a blue ball. If the red ball changes its position slightly, it may fall on the other side of the green line. Similarly, if the blue ball changes its position slightly, it may be misclassified. Both the green and red lines are more sensitive to small changes in the observations. The black line on the other hand is less sensitive and less susceptible to model variance.

In an n -dimensional space, a hyperplane is a flat subspace of dimension $n - 1$. For example, in two dimensions a straight line is a one-dimensional hyperplane, as shown in the Figure 3. In three dimensions, a hyperplane is a flat two-dimensional subspace, i.e. a plane. Mathematically in n dimensions a separating hyperplane is a linear combination of all dimensions equated to 0; i.e.,

$$\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n = 0$$

The scalar θ_0 is often referred to as a bias. If $\theta_0 = 0$, then the hyperplane goes through the origin.

A hyperplane acts as a separator. The points lying on two different sides of the hyperplane will make up two different groups.

Basic idea of support vector machines is to find out the optimal hyperplane for linearly separable patterns. A natural choice of separating hyperplane is optimal margin hyperplane (also known as optimal separating hyperplane) which is farthest from the observations. The perpendicular distance from each observation to a given separating hyperplane is computed. The smallest of all those distances is a measure of how close the hyperplane is to the group of observations. This minimum distance is known as the margin. The operation of the SVM algorithm is based on finding the hyperplane that gives the largest minimum distance to the training examples, i.e. to find the maximum margin. This is known as the maximal margin classifier.

A separating hyperplane in two dimension can be expressed as

$$\theta_0 + \theta_1 x_1 + \theta_2 x_2 = 0$$

Hence, any point that lies above the hyperplane, satisfies

$$\theta_0 + \theta_1 x_1 + \theta_2 x_2 > 0$$

and any point that lies below the hyperplane, satisfies

$$\theta_0 + \theta_1 x_1 + \theta_2 x_2 < 0$$

The coefficients or weights θ_1 and θ_2 can be adjusted so that the boundaries of the margin can be written as

$$H_1: \theta_0 + \theta_1 x_{1i} + \theta_2 x_{2i} \geq 1, \text{ for } y_i = +1$$

$$H_2: \theta_0 + \theta_1 x_{1i} + \theta_2 x_{2i} \leq -1, \text{ for } y_i = -1$$

This is to ascertain that any observation that falls on or above H_1 belongs to class +1 and any observation that falls on or below H_2 , belongs to class -1. Alternatively, we may write

$$y_i(\theta_0 + \theta_1 x_{1i} + \theta_2 x_{2i}) \geq 1 \text{ for every observation}$$

The boundaries of the margins, H_1 and H_2 , are themselves hyperplanes too. The training data that falls exactly on the boundaries of the margin are called the support vectors as they support the maximal margin hyperplane in the sense that if these

points are shifted slightly, then the maximal margin hyperplane will also shift.

Note that the maximal margin hyperplane depends directly only on these support vectors.

If any of the other points change, the maximal margin hyperplane does not change, until the movement affects the boundary conditions or the support vectors. The support vectors are the most difficult to classify and give the most information regarding classification. Since the support vectors lie on or closest to the decision boundary, they are the most essential or critical data points in the training set.

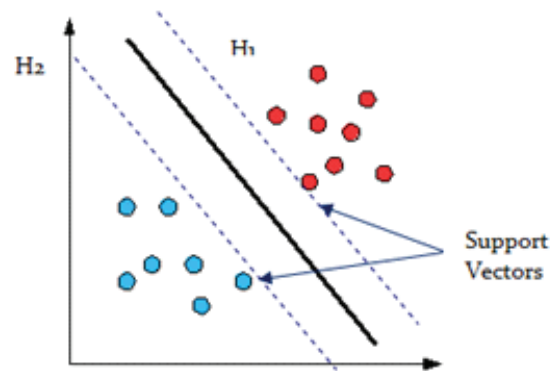


Figure 3: Support Vectors

For a general n -dimensional feature space, the defining equation becomes:

$$y_i(\theta_0 + \theta_1 x_{2i} + \theta_2 x_{2i} + \dots + \theta_n x_{ni}) \geq 1, \text{ for every observation}$$

If the vector of the weights is denoted by Θ and $|\Theta|$ is the norm of this vector, then it is easy to see that the size of the maximal margin is $2/|\Theta|$. Finding the maximal margin hyperplanes and support vectors is a problem of convex quadratic optimization. It is important to note that the complexity of SVM is characterized by the number of support vectors, rather than the dimension of the feature space. That is the reason SVM has a comparatively less tendency to overfit. If all data points other than the support vectors are removed from the training data set, and the training algorithm is repeated, the same separating hyperplane would be found. The number of support vectors provides an upper bound to the expected error rate of the SVM classifier, which happens to be independent of data dimensionality. An SVM with a small number of support vectors has good generalization, even when the data has high dimensionality.

10.6.2 Super Vector Classifier

The maximal margin classifier is a very natural way to perform classification, is a separating hyperplane exists. However existence of such a hyperplane may not be guaranteed, or even if it exists, the data is noisy so that maximal margin classifier provides a poor solution. In such cases, the concept can be extended where a hyperplane exists which almost separates the classes, using what is known as a soft margin. The generalization of the maximal margin classifier to the non-separable case is known as the support vector classifier, where a small proportion of the training sample is allowed to cross the margins, or even the separating hyperplane. Rather than looking for the largest possible margin so that every observation is on the correct side of the margin, thereby making the margins very narrow or non-existent, some observations are allowed to be on the incorrect side of the

margins. The margin is soft as a small number of observations violate the margin. The softness is controlled by slack variables which control the position of the observations relative to the margins and separating hyperplane. The support vector classifier maximizes a soft margin. The optimization problem can be modified as:

$$y_i(\theta_0 + \theta_1 x_{1i} + \theta_2 x_{2i} + \dots + \theta_n x_{ni}) \geq 1 - \epsilon_i \text{ for every observation}$$

$$\text{Where } \epsilon_i \geq 0 \text{ and } \sum_{i=1}^n \epsilon_i \leq C$$

The ϵ_i is the slack corresponding to i -th observation and C is a regularization parameter set by user. Larger value of C leads to larger penalty for errors.

However there will be situations when a linear boundary simply does not work.

10.6.3 When Data is Not Linearly Separable

SVM is quite intuitive when the data is linearly separable. However, when they are not, as shown in the Figure 4 below, SVM can be extended to perform well.

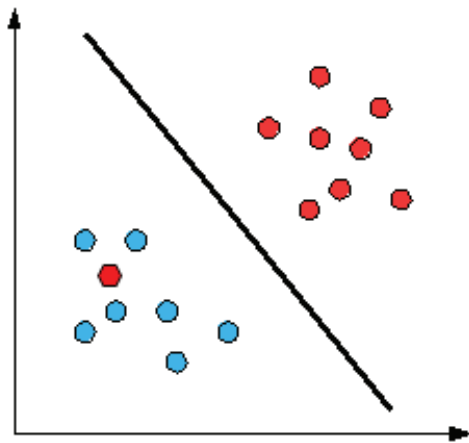


Figure 4: Linearly Non Separable

There are two main steps for nonlinear generalization of SVM. The first step involves transformation of the original training (input) data into a higher dimensional data using a nonlinear mapping. Once the data is transformed into the new higher dimension, the second step involves finding a linear separating hyperplane in the new space. The maximal marginal hyperplane found in the new space corresponds to a nonlinear separating hyper-surface in the original space.

Let us study Rule-based classification technique in the next section.

10.7 RULE-BASED CLASSIFICATION

Rule-based classifier makes use of a set of IF-THEN rules for classification. We can express a rule in the following form

IF condition THEN conclusion

Let us consider a rule R1,

R1: IF age = youth AND student = yes

THEN buy_computer = yes

Rule-Based Classifier classifies records by using a collection of “if...then...” rules.

Rule Notation: *(Condition) → Class Label*

Example 1:

$(\text{BloodType} = \text{Warm}) \wedge (\text{LayEggs} = \text{Yes}) \rightarrow \text{Birds}$

$(\text{TaxableIncome} < 50\text{K}) \vee (\text{Refund} = \text{Yes}) \rightarrow \text{Evade} = \text{No}$

Example 2:

Name	Blood Type	Give Birth	Can Fly	Live in Water	Class
human	warm	yes	no	no	mammals
python	cold	no	no	no	reptiles
salmon	cold	no	no	yes	fishes
whale	warm	yes	no	yes	mammals
frog	cold	no	no	sometimes	amphibians
komodo	cold	no	no	no	reptiles
bat	warm	yes	yes	no	mammals
pigeon	warm	no	yes	no	birds
cat	warm	yes	no	no	mammals
leopard shark	cold	yes	no	yes	fishes
turtle	cold	no	no	sometimes	reptiles
penguin	warm	no	no	sometimes	birds
porcupine	warm	yes	no	no	mammals
eel	cold	no	no	yes	fishes
salamander	cold	no	no	sometimes	amphibians
gila monster	cold	no	no	no	reptiles
platypus	warm	no	no	no	mammals
owl	warm	no	yes	no	birds
dolphin	warm	yes	no	yes	mammals
eagle	warm	no	yes	no	birds

R1: $(\text{Give Birth} = \text{no}) \& (\text{Can Fly} = \text{yes}) \rightarrow \text{Birds}$

R2: $(\text{Give Birth} = \text{no}) \& (\text{Live in Water} = \text{yes}) \rightarrow \text{Fishes}$

R3: $(\text{Give Birth} = \text{yes}) \& (\text{Blood Type} = \text{warm}) \rightarrow \text{Mammals}$

R4: $(\text{Give Birth} = \text{no}) \& (\text{Can Fly} = \text{no}) \rightarrow \text{Reptiles}$

R5: $(\text{Live in Water} = \text{sometimes}) \rightarrow \text{Amphibians}$

10.7.1 Application of Rule-Based Classifier

A rule **r** covers an instance **x** if the attributes of the instance satisfy the condition of the rule.

Name	Blood Type	Give Birth	Can Fly	Live in Water	Class
hawk	warm	no	yes	no	?
grizzly bear	warm	yes	no	no	?

The rule **R1** above covers a **hawk** => **Bird**

The rule **R3** covers the **grizzly bear** => **Mammal**

Name	Blood Type	Give Birth	Can Fly	Live in Water	Class
lemur	warm	yes	no	no	?
turtle	cold	no	no	sometimes	?
dogfish shark	cold	yes	no	yes	?

A **lemur** triggers rule R3, so it is classified => Mammal

A **turtle** triggers both R4 and R5

A **dogfish shark** trigger matches none of the rules.

10.7.2 Advantages of Rule-Based Classifiers

Following are the advantages of Rule-Based Classifiers:

- They are as highly expressive as decision trees -Easy to interpret
- They are easy to generate
- They can classify new instances rapidly
- Their performance is comparable to decision trees

10.7.3 Coverage and Accuracy

Go through the below given example to know about the coverage and accuracy,

Example 3:

<i>Tid</i>	Refund	Marital Status	Taxable Income	Class
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Coverage of a rule:

- Fraction of all records that satisfy the antecedent of a rule
- $\text{Count}(\text{instances with antecedent}) / \text{Count}(\text{training set})$
- Example on left: (Status = 'Single') $\rightarrow$ no, Coverage = $4/10 = 40\%$

Accuracy of a rule:

- Fraction of records that satisfy the antecedent that also satisfy the consequent of a rule
- $\text{Count}(\text{instances with antecedent AND consequent}) / \text{Count}(\text{instances with antecedent})$
- Example on left: (Status = 'Single') $\rightarrow$ no, accuracy = $2/4 = 50\%$

Important points to be remembered are:

- The Left Hand Side is **rule antecedent or condition**
- The Right Hand Side is **rule consequent**
- **Coverage** of a rule - Fraction of records that satisfy the antecedent of a rule

- **Accuracy** of a rule - Fraction of records that satisfy both the antecedent and consequent of a rule.

10.7.4 Characteristics of Rule Sets

If we convert the result of decision tree to classification rules, these rules would be mutually exclusive and exhaustive at the same time.

Mutually Exclusive rules

- Classifier contains mutually exclusive rules if the rules are independent of each other.
- Every record is covered by at most one rule.

Exhaustive rules

- Classifier has exhaustive coverage if it accounts for every possible combination of attribute values.
- Each record is covered by at least one rule.

These rules can be simplified. However, simplified rules may no longer be mutually exclusive since a record may trigger more than one rule. Simplified rules may no longer be exhaustive either since a record may not trigger any rules.

Handling rules that are not mutually exclusive

- A record may trigger more than one rule
- Solutions for these are:
 - o Ordered rule set
 - o Unordered rule set – use voting schemes

Handling rules that are not exhaustive

- A record may not trigger any rules
- Solution for this is
 - o Use a default class

10.7.5 Ordered Rule Set

An ordered rule set is known as a decision list. Rules are rank ordered according to their priority. For example, when a test record is presented to the classifier, it is assigned to the class label of the highest ranked rule it has triggered. If none of the rules fired, it is assigned to the default class.

From the table given in the Example 2,

1. R1: (Give Birth = no) & (Can Fly = yes) → Birds
2. R2: (Give Birth = no) & (Live in Water = yes) → Fishes
3. R3: (Give Birth = yes) & (Blood Type = warm) → Mammals
4. R4: (Give Birth = no) & (Can Fly = no) → Reptiles
5. R5: (Live in Water = sometimes) → Amphibians

A turtle triggers both R4 and R5, but by the order, the conclusion is =>Reptile

10.7.6 Rule Ordering Schemes

If more than one rule is triggered, it needs conflict resolution in the following ways:

- **Size ordering** - assign the highest priority to the triggering rules that has the “toughest” requirement (i.e., with the most attribute test)
- **Class-based ordering** - decreasing order of prevalence or misclassification cost per class
- **Rule-based ordering** (decision list) - rules are organized into one long priority list, according to some measure of rule quality or by experts

Let us see the table given below:

<i>Tid</i>	Refund	Marital Status	Taxable Income	Class
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Rule-based Ordering

(Refund=Yes) ==> No

(Refund=No, Marital Status={Single,Divorced}, Taxable Income<80K) ==> No

(Refund=No, Marital Status={Single,Divorced}, Taxable Income>80K) ==> Yes

(Refund=No, Marital Status={Married}) ==> No

Class-based Ordering

(Refund=Yes) ==> No

(Refund=No, Marital Status={Single,Divorced}, Taxable Income<80K) ==> No

(Refund=No, Marital Status={Married}) ==> No

(Refund=No, Marital Status={Single,Divorced}, Taxable Income>80K) ==> Yes

- R1: Accuracy = 100%, coverage = 30%
- R2: Accuracy = 100%, coverage = 10%
- R3: Accuracy = 100%, coverage = 30%
- R4: Accuracy = 100%, coverage = 30%

10.7.7 Building Classification Rules

There are two methods to build the classification rules as shown below:

10.7.7.1 Direct Method

In direct method, rules are extracted directly from data

Examples: Sequential-Covering such as RIPPER, CN2, Holte's 1R

10.7.7.2 Indirect Method

In the Indirect method, rules are extracted from other classification models (e.g. decision trees, neural networks, etc).

Example: C4.5 Rules

Direct Method extract rules directly from data. Sequential Covering such as CN2 Algorithm and RIPPER Algorithm are common direct methods for building classification rules.

For 2-class problem, choose one of the classes as positive class, and the other as negative class. You may choose the positive class as the smaller number of cases.

- Learn rules for the positive class
- Negative class becomes the default class.

To generalize for multi-class problem,

- Order the classes according to increasing class prevalence (fraction of instances that belong to a particular class),
- Learn the rule set for smallest class first, treat the rest as negative class.
- Repeat with next smallest class as positive class.
- Thus the largest class will become the default class.

Sequential-Covering in a Nut-shell

- Start from an empty rule
- Grow a rule using the Learn-One-Rule function (Rule Growing)
- Remove training records covered by the rule (Instance Elimination)
- Repeat Step (2) and (3) until stopping criterion is met
- (Optional) Rule Pruning

More formally, sequential-covering can be represented as:

Sequential-Covering (Target_attribute, Attributes, Examples, Threshold)

- Learned_rules $\leftarrow$ { }
- Rule $\leftarrow$ Learn-one-rule(Target_attribute, Attributes, Examples)
- While Performance(Rule, Examples) > Threshold :
 - o Learned_rules $\leftarrow$ Learned_rules + Rule
 - o Examples $\leftarrow$ Examples - {examples correctly classified by Rule}
 - o Rule $\leftarrow$ Learn-One-Rule (Target_attribute, Attributes, Examples)
- Learned_rules $\leftarrow$ sort Learned_rules according to Performance over Examples

- Return Learned_rules

However, because Sequential-Covering does not backtrack, this algorithm is not guaranteed to find the smallest or best set of rules. Thus the Learn-One-Rule function must be very effective.

Rule Growing

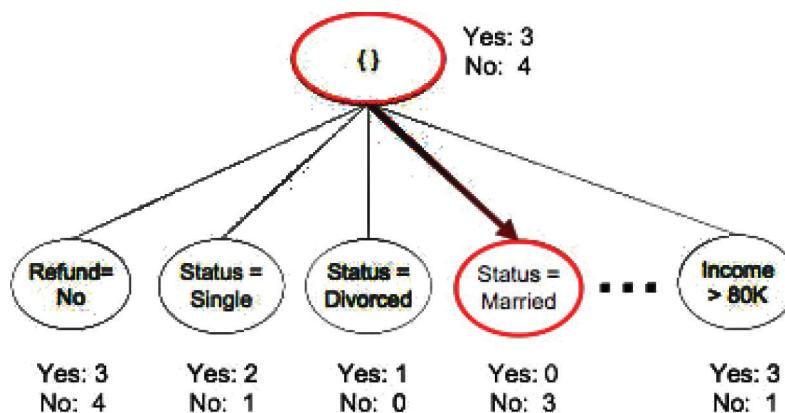
In Rule Growing, there are two approaches namely, Top-down and Bottom-up.

Top-down -- General-to-specific

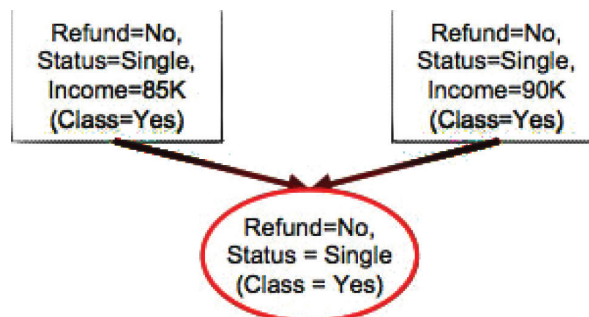
Example at Figure 5(a) shows a specific rule determining 3 cases, all classified as 'No'. Choice is picked because of the number of cases and accuracy.

Bottom-up -- Specific-to-general

Example at Figure 5(b) shows two rules (top) and the income part doesn't distinguish (85K or 90K) and thus the rule can be simplified.



(a) General-to-specific



(b) Specific-to-general

Figure 5: (a) General-to-specific (b) Specific-to-general

Rule Evaluation

Rule evaluation is all about how do we determine the best next rule.

First Order Inductive Learner's (FOIL) Information Gain is an early rule based learning algorithm.

R0: {} => class (initial rule)

R1: {A} => class (rule after adding conjunct--a condition component in the antecedent)

Information Gain(R_0, R_1) = $t * [\log (p_1/(p_1+n_1)) - \log (p_0/(p_0 + n_0))]$

where,

t = number of positive instances covered by both R_0 and R_1

p_0 = number of positive instances covered by R_0

n_0 = number of negative instances covered by R_0

p_1 = number of positive instances covered by R_1

n_1 = number of negative instances covered by R_1

Gain(R_0, R_1) is similar to the entropy gain calculations used in decision trees.

Instance Elimination

Following are the criteria to be investigated:

- Why do we need to eliminate instances? Otherwise, the next rule is identical to previous rule
- Why do we remove positive instances? Ensure that the next rule is different
- Why do we remove negative instances? Prevent underestimating accuracy of rule

Direct Method: RIPPER (Repeated Incremental Pruning to Produce Error Reduction)

For 2-class problem, choose one of the classes as positive class, and the other as negative class. You may choose the positive class as the smaller number of cases.

- Learn rules for positive class
- Negative class become the default class

To generalize for multi-class problem,

- Order the classes according to increasing class prevalence (fraction of instances that belong to a particular class)
- Learn the rule set for smallest class first, treat the rest as negative class
- Repeat with next smallest class as positive class
- Thus the largest class will become the default class

Growing a Rule

- Start from empty rule
- Repeat: Add conjuncts as long as they improve FOIL's information gain
 - o Stop when rule no longer covers negative examples
 - o We can get a rather extensive rule such as $ABCD \rightarrow y$
- Prune the rule immediately using incremental reduced error pruning
 - o Measure for pruning: $v = (p-n)/(p+n)$
 - p : number of positive examples covered by the rule in the validation set

- n : number of negative examples covered by the rule in the validation set
- o Pruning method: delete any final sequence of conditions that maximizes v
- o Example: if the grown rule is $ABCD \rightarrow y$, check to prune D then CD , then BCD

Building a Rule Set

Uses the sequential covering algorithm

- Finds the best rule that covers the current set of positive examples
- Eliminate both positive and negative examples covered by the rule

Each time a rule is added to the rule set, compute the new description length

- Stop adding new rules when the new description length is d bits longer than the smallest description length obtained so far (default setting for $d=64$ bits) [this is not clear from the text]
- Alternatively stop when the error rate exceeds 50%

Optimize the rule set:

- For each rule r in the rule set R
 - o Consider 2 alternative rules:
 - Replacement rule (r^*): grow new rule from scratch
 - Revised rule(r'): add conjuncts to extend the rule r
 - o Compare the rule set for r against the rule set for r^* and r'
 - o Choose rule set that minimizes MDL principle (minimum description length-- a measure of model complexity)
 - Repeat rule generation and rule optimization for the remaining positive examples

Indirect Method: C4.5 Rules

Extract rules from an unpruned decision tree

For each rule, $r: A \rightarrow y$,

- consider an alternative rule $r': A' \rightarrow y$ where A' is obtained by removing one of the conjuncts in A
- Compare the pessimistic error rate for r against all r' 's
- Prune if one of the alternative rules has lower pessimistic error rate
- Repeat until we can no longer improve generalization error

Instead of ordering the rules, order subsets of rules (class ordering)

- Each subset is a collection of rules with the same rule consequent (class)
- Compute description length of each subset

- o Description length = $L(\text{error}) + g * L(\text{model})$
- o g is a parameter that takes into account the presence of redundant attributes in a rule set (default value = 0.5)
- o Similar to the generalization error calculation of a decision tree

10.7.8 Advantages of Rule-Based Classifiers

Following are some of the advantages of Rule-Based classifiers:

- They have the characteristics quite similar to decision trees
- These classifiers are as highly expressive as decision trees
- They are easy to interpret
- Their performance is comparable to decision trees
- They can handle redundant attributes
- They are better suited for handling imbalanced classes
- There are harder to handle missing values in the test set

Evaluating the performance of a data mining technique is a fundamental aspect of data mining. In the following section let us focus on this aspect.

10.8 MODEL EVALUATION AND SELECTION

Evaluating the performance of a data mining technique is a fundamental aspect of data mining. Evaluation method is the yardstick to examine the efficiency and performance of any model. The evaluation is important for understanding the quality of the model or technique, for refining parameters in the iterative process of learning and for selecting the most acceptable model or technique from a given set of models or techniques. There are several criteria for evaluating models for different tasks and other criteria that can be important as well, such as the computational complexity or the comprehensibility of the model.

Model selection is a technique for selecting the best model after the individual models are evaluated based on the required criteria. Model selection is the problem of choosing one from among a set of candidate models. It is common to choose a model that performs the best on a hold-out test dataset or to estimate model performance using a resampling technique, such as k-fold cross-validation.

An alternative approach to model selection involves using probabilistic statistical measures that attempt to quantify both the model performance on the training dataset and the complexity of the model. Examples include the Akaike (AIC) and Bayesian Information Criterion(BIC) and the Minimum Description Length (MDL).

The benefit of these information criterion statistics is that they do not require a hold-out test set, although a limitation is that they do not take the uncertainty of the models into account and may end-up selecting models that are too simple.

Model evaluation is a method of assessing the correctness of models on test data. The test data consists of data points that have not been seen by the model before.

10.8.1 Types of Model Selection

There are many common approaches that may be used for model selection. For example, in the case of supervised learning, the three most common approaches are:

- Train, Validation, and Test datasets
- Resampling Methods
- Probabilistic Statistics

The simplest reliable method of model selection involves fitting candidate models on a training set, tuning them on the validation dataset, and selecting a model that performs the best on the test dataset according to a chosen metric, such as accuracy or error. A problem with this approach is that it requires a lot of data.

Resampling techniques attempt to achieve the same as the train/val/test approach to model selection, although using a small dataset. An example is k-fold cross-validation where a training set is split into many train/test pairs and a model is fit and evaluated on each. This is repeated for each model and a model is selected with the best average score across the k-folds. A problem with this and the prior approach is that only model performance is assessed, regardless of model complexity.

A third approach to model selection attempts to combine the complexity of the model with the performance of the model into a score, then select the model that minimizes or maximizes the score.

Resampling methods

Resampling methods, as the name suggests, are simple techniques of rearranging data samples to inspect if the model performs well on data samples that it has not been trained on. In other words, resampling helps us understand if the model will generalize well.

Cross-validation

Cross-validation is a technique for estimating the generalization performance of a predictive model. The main idea behind CV is to split data, once or several times, for estimating the risk of each algorithm: Part of data (the training sample) is used for training each algorithm, and the remaining part (the validation sample) is used for estimating the risk of the algorithm. Then, CV selects the algorithm with the smallest estimated risk. Cross validation is an alternative to random sub-sampling.

Holdout Method

Hold-out or (simple) validation relies on a single split of data. The holdout method is the simplest kind of cross validation. The data set is separated into two sets, called the training set and the testing set. The function approximator fits a function using the training set only. Then the function approximator is asked to predict the output values for the data in the testing set (it has never seen these output values before). The errors it makes are accumulated as before to give the mean absolute test set error, which is used to evaluate the model. The advantage of this method is that it is usually preferable to the residual method and takes no longer to compute. However, its evaluation can have a high variance. The evaluation may depend heavily on which data points end up in the training set and which end up in the test set, and thus the evaluation may be significantly different depending on how the

division is made.

Random Sub-sampling

The hold out method can be repeated several times to improve the estimation of a classifier's performance. This approach is known as random sub-sampling. Random sub-sampling encounters some of the problems associated with the holdout method because it does not utilize as much data as possible for training. It has also no control over the number of times each record is used for testing and training. Consequently, some records might be used for training more often than others.

k-fold Cross-validation

It is one way to improve over the holdout method. The data set is divided into k subsets, and the holdout method is repeated k times. Each time, one of the k subsets is used as the test set and the other $k-1$ subsets are put together to form a training set. Then the average error across all k trials is computed. The advantage of this method is that it matters less how the data gets divided. Every data point gets to be in a test set exactly once, and gets to be in a training set $k-1$ times. The variance of the resulting estimate is reduced as k is increased. The disadvantage of this method is that the training algorithm has to be rerun from scratch k times, which means it takes k times as much computation to make an evaluation. A variant of this method is to randomly divide the data into a test and training set k different times. The advantage of doing this is that you can independently choose how large each test set is and how many trials you average over.

Leave-one-out Method

When K -fold cross-validation taken to its logical extreme, with K equal to N , the number of data points in the set. That means that N separate times, the function approximator is trained on all the data except for one point and a prediction is made for that point. As before the average error is computed and used to evaluate the model. The evaluation given by leave-one-out cross validation error (LOO-XVE) is good, but at first pass it seems very expensive to compute. Fortunately, locally weighted learners can make LOO predictions just as easily as they make regular predictions. That means computing the LOO-XVE takes no more time than computing the residual error and it is a much better way to evaluate models.

Random Split

Random Splits are used to randomly sample a percentage of data into training, testing, and preferably validation sets. The advantage of this method is that there is a good chance that the original population is well represented in all the three sets. In more formal terms, random splitting will prevent a biased sampling of data. It is very important to note the use of the validation set in model selection. The validation set is the second test set and one might ask, why have two test sets?

In the process of feature selection and model tuning, the test set is used for model evaluation. This means that the model parameters and the feature set are selected such that they give an optimal result on the test set. Thus, the validation set which has completely unseen data points (not been used in the tuning and feature selection modules) is used for the final evaluation.

Time-Based Split

There are some types of data where random splits are not possible. For example, if

we have to train a model for weather forecasting, we cannot randomly divide the data into training and testing sets. This will jumble up the seasonal pattern. Such data is often referred to by the term, Time Series.

In such cases, a time-wise split is used. The training set can have data for the last three years and 10 months of the present year. The last two months can be reserved for the testing or validation set.

There is also a concept of window sets, where the model is trained till a particular date and tested on the future dates iteratively such that the training window keeps increasing shifting by one day (consequently, the test set also reduces by a day). The advantage of this method is that it stabilizes the model and prevents overfitting when the test set is very small, for example, 3 to 7 days.

However, the drawback of time-series data is that the events or data points are not mutually independent. One event might affect every data input that follows after.

K-Fold Cross-Validation

The cross-validation technique works by randomly shuffling the dataset and then splitting it into k groups. Thereafter, on iterating over each group, the group needs to be considered as a test set while all other groups are clubbed together into the training set. The model is tested on the test group and the process continues for k groups.

Thus, by the end of the process, one has k different results on k different test groups. The best model can then be selected easily by choosing the one with the highest score.

Stratified K-Fold

The process for stratified K-Fold is similar to that of K-Fold cross-validation with one single point of difference, unlike in k -fold cross-validation, the values of the target variable is taken into consideration in stratified k -fold.

If for instance, the target variable is a categorical variable with 2 classes, then stratified k -fold ensures that each test fold gets an equal ratio of the two classes when compared to the training set.

This makes the model evaluation more accurate and the model training less biased.

Bootstrap

Bootstrap is one of the most powerful ways to obtain a stabilized model. It is close to the random splitting technique since it follows the concept of random sampling.

The first step is to select a sample size (which is usually equal to the size of the original dataset). Thereafter, a sample data point must be randomly selected from the original dataset and added to the bootstrap sample. After the addition, the sample needs to be put back into the original sample. This process needs to be repeated for N times, where N is the sample size.

Therefore, it is a resampling technique that creates the bootstrap sample by sampling data points from the original dataset with replacement. This means that the bootstrap sample can contain multiple instances of the same data point. The model is trained on the bootstrap sample and then evaluated on all those data points that did not make it to the bootstrapped sample. These are called the out-of-bag samples.

Probabilistic measures

Probabilistic Measures do not just take into account the model performance but also the model complexity. Model complexity is the measure of the model's ability to capture the variance in the data.

For example, a highly biased model like the linear regression algorithm is less complex and on the other hand, a neural network is very high on complexity.

Another important point to note here is that the model performance taken into account in probabilistic measures is calculated from the training set only. A hold-out test set is typically not required.

A fair bit of disadvantage however lies in the fact that probabilistic measures do not consider the uncertainty of the models and has a chance of selecting simpler models over complex models.

There are three statistical approaches to estimating how well a given model fits a dataset and how complex the model is. And each can be shown to be equivalent or proportional to each other, although each was derived from a different framing or field of study.

They are:

- Akaike Information Criterion (AIC). Derived from frequentist probability
- Bayesian Information Criterion (BIC). Derived from Bayesian probability
- Minimum Description Length (MDL). Derived from information theory

Akaike Information Criterion (AIC)

It is common knowledge that every model is not completely accurate. There is always some information loss which can be measured using the KL information metric. Kulback-Liebler or KL divergence is the measure of the difference in the probability distribution of two variables.

A statistician, Hirotugu Akaike, took into consideration the relationship between KL Information and Maximum Likelihood (in maximum-likelihood, one wishes to maximize the conditional probability of observing a datapoint X, given the parameters and a specified probability distribution) and developed the concept of Information Criterion (or IC). Therefore, Akaike's IC or AIC is the measure of information loss. This is how the discrepancy between two different models is captured and the model with the least information loss is suggested as the model of choice.

$$AIC = (2K - 2\log(L))/N$$

where,

K = number of independent variables or predictors

L = maximum-likelihood of the model

N = number of data points in the training set (especially helpful in case of small datasets)

The limitation of AIC is that it is not very good with generalizing models as it tends to select complex models that lose less training information.

Bayesian Information Criterion (BIC)

BIC was derived from the Bayesian probability concept and is suited for models that are trained under the maximum likelihood estimation.

where,

K = number of independent variables

L = maximum-likelihood

N = Number of sampler/data points in the training set

BIC penalizes the model for its complexity and is preferably used when the size of the dataset is not very small (otherwise it tends to settle on very simple models).

Minimum Description Length (MDL)

MDL is derived from the Information theory which deals with quantities such as entropy that measure the average number of bits required to represent an event from a probability distribution or a random variable.

MDL or the minimum description length is the minimum number of such bits required to represent the model.

$$MDL = L(h) + L(D|h)$$

where,

d = model

D = predictions made by the model

$L(h)$ = number of bits required to represent the model

$L(D | h)$ = number of bits required to represent the predictions from the model

10.8.2 Models Evaluation

Models can be evaluated using multiple metrics. However, the right choice of an evaluation metric is crucial and often depends upon the problem that is being solved. A clear understanding of a wide range of metrics can help the evaluator to chance upon an appropriate match of the problem statement and a metric.

Classification metrics

For every classification model prediction, a matrix called the confusion matrix can be constructed which demonstrates the number of test cases correctly and incorrectly classified.

Confusion Matrix

A binary classification model classifies each instance into one of two classes; say a true and a false class. This gives rise to four possible classifications for each instance: a true positive, a true negative, a false positive, or a false negative. This situation can be depicted as a confusion matrix (also called contingency table) given in Figure 6. The confusion matrix juxtaposes the observed classifications for a phenomenon (columns) with the predicted classifications of a model (rows). In Figure 6, the classifications that lie along the major diagonal of the table are the correct classifications, that is, the true positives and the true negatives. The other fields signify model errors. For a perfect model we would only see the true

positive and true negative fields filled out, the other fields would be set to zero. It is common to call true positives hits, true negatives correct rejections, false positive false alarms, and false negatives misses. A number of model performance metrics can be derived from the confusion matrix. Perhaps, the most common metric is accuracy.

It looks something like this (considering 1 –True/Positive and 0 –False/Negative are the target classes):

		Observed	
		True	False
Predicted	True	True Positive (TP)	False Positive (FP)
	False	False Negative (FN)	True Negative (TN)

Figure 6: Format of Confusion Matrix

- TN: Number of negative cases correctly classified
- TP: Number of positive cases correctly classified
- FN: Number of positive cases incorrectly classified as negative
- FP: Number of negative cases correctly classified as positive

Accuracy

Accuracy is the simplest metric and can be defined as the number of test cases correctly classified divided by the total number of test cases.

It can be applied to most generic problems but is not very useful when it comes to unbalanced datasets.

For instance, if we are detecting frauds in bank data, the ratio of fraud to non-fraud cases can be 1:99. In such cases, if accuracy is used, the model will turn out to be 99% accurate by predicting all test cases as non-fraud. The 99% accurate model will be completely useless.

If a model is poorly trained such that it predicts all the 1000 (say) data points as non-frauds, it will be missing out on the 10 fraud data points. If accuracy is measured, it will show that that model correctly predicts 990 data points and thus, it will have an accuracy of $(990/1000) \times 100 = 99\%$.

This is why accuracy is a false indicator of the model's health.

Therefore, for such a case, a metric is required that can focus on the ten fraud data points which were completely missed by the model.

Precision

Precision is the metric used to identify the correctness of classification.

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Intuitively, this equation is the ratio of correct positive classifications to the total number of predicted positive classifications. The greater the fraction, the higher is the precision, which means better is the ability of the model to correctly classify the positive class. In the problem of predictive maintenance (where one must predict in advance when a machine needs to be repaired), precision comes into play. The cost of maintenance is usually high and thus, incorrect predictions can lead to a loss for the company. In such cases, the ability of the model to correctly classify the positive class and to lower the number of false positives is paramount.

Recall

Recall tells us the number of positive cases correctly identified out of the total number of positive cases.

Going back to the fraud problem, the recall value will be very useful in fraud cases because a high recall value will indicate that a lot of fraud cases were identified out of the total number of frauds.

F1- Score

F1- score is also known as F-Score is the harmonic mean of Recall and Precision and therefore, balances out the strengths of each. It is useful in cases where both recall and precision can be valuable – like in the identification of plane parts that might require repairing. Here, precision will be required to save on the company's cost (because plane parts are extremely expensive) and recall will be required to ensure that the machinery is stable and not a threat to human lives.

$$F1Score = 2 * ((precision * recall) / (precision + recall))$$

G-score is the geometric mean of precision and recall:

$$G = \sqrt{precision \cdot recall}$$

Receiver Operating Curves (ROC)

Central to constructing, deploying, and using classification models is the question of model performance assessment. Traditionally this is accomplished by using metrics derived from the confusion matrix or contingency table. However, it has been recognized that (a) a scalar is a poor summary for the performance of a model in particular when deploying non- parametric models such as artificial neural networks or decision trees and (b) some performance metrics derived from the confusion matrix are sensitive to data anomalies such as class skew.

It has been observed that Receiver Operating Characteristic (ROC) curves visually convey the same information as the confusion matrix in a much more intuitive and robust fashion. ROC curves are two-dimensional graphs that visually depict the performance and performance trade-off of a classification model. ROC curves were originally designed as tools in communication theory to visually determine optimal operating points for signal discriminators.

Two new performance metrics have to be introduced here in order to construct ROC curves (they have been defined here in terms of the confusion matrix), the true positive rate (TPR) and the false positive rate (FPR):

$$\text{True positive rate} = TP / (TP + FN) = 1 - \text{false negative rate}$$

$$\text{False positive rate} = FP / (FP + TN) = 1 - \text{true negative rate}$$

ROC graphs are constructed by plotting the true positive rate against the false positive rate (figure 7 (a)). A number of regions of interest in a ROC graph can be identified. The diagonal line from the bottom left corner to the top right corner denotes random classifier performance, that is, a classification model mapped onto this line produces as many false positive responses as it produces true positive responses.

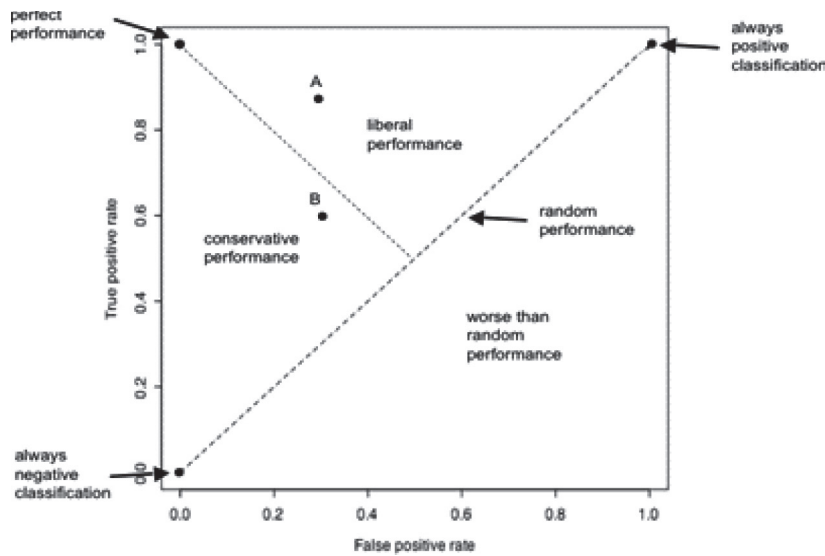
To the left bottom of the random performance line there is the conservative performance region. Classifiers in this region commit few false positive errors. In the extreme case, denoted by point in the bottom left corner, a conservative classification model will classify all instances as negative. In this way it will not commit any false positives but it will also not produce any true positives. The region of classifiers with liberal performance occupies ROC graphs are constructed by plotting the true positive rate against the false positive rate (figure 7(a)).

A number of regions of interest can be identified in a ROC graph. The diagonal line from the bottom left corner to the top right corner denotes random classifier performance, that is, a classification model mapped onto this line produces as many false positive responses as it produces true positive responses. To the left bottom of the random performance line is the conservative performance region. Classifiers in this region commit few false positive errors.

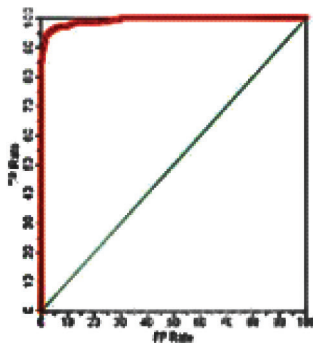
In the extreme case, denoted by point in the bottom left corner, a conservative classification model will classify all instances as negative. In this way it will not commit any false positives but it will also not produce any true positives. The region of classifiers with liberal performance occupies the top of the graph. These classifiers have a good true positive rate but also commit substantial numbers of false positive errors.

Again, in the extreme case denoted by the point in the top right corner, we have classification models that classify every instance as positive. In that way, the classifier will not miss any true positives but it will also commit a very large number of false positives. Classifiers that fall in the region to the right of the random performance line have a performance worse than random performance, that is, they consistently produce more false positive responses than true positive responses.

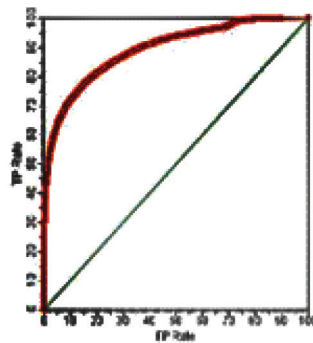
However, because ROC graphs are symmetric along the random performance line, inverting the responses of a classifier in the “worse than random performance” region will turn it into a well performing classifier in one of the regions above the random performance line. Finally, the point in the top left corner denotes perfect classification: 100% true positive rate and 0% false positive rate.



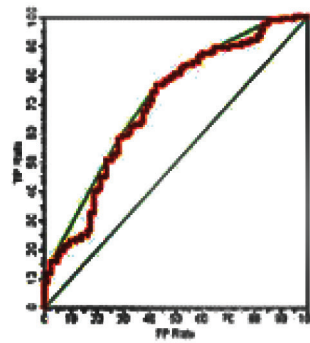
(a)



(b)



(c)



(d)

Figure 7: ROC curves: (a) Regions of a ROC graph (b) An almost perfect classifier (c) A reasonable classifier (d) A poor classifier

The point marked with A is the classifier from the previous section with a TPR = 0.90 and a FPR = 0.35. Note, that the classifier is mapped to the same point in the ROC graph regardless whether we use the original test set or the test set with the sampled down negative class illustrating the fact that ROC graphs are not sensitive to class skew. Classifiers mapped onto a ROC graph can be ranked according to their distance to the ‘perfect performance’ point. In Figure 2 (a) classifier A is considered to be superior to a hypothetical classifier B because A is closer to the top left corner.

The true power of ROC curves, however, comes from the fact that they characterize the performance of a classification model as a curve rather than a single point on the ROC graph. In addition, Figure 2 shows some typical examples of ROC curves. Part (b) depicts the ROC curve of an almost perfect classifier where the performance curve almost touches the ‘perfect performance’ point in the top left corner. Part (c) and part (d) depict ROC curves of inferior classifiers. At this level the curves provide a convenient visual representation of the performance of various models where it is easy to spot optimal versus sub-optimal models.

Log Loss

Log loss is a very effective classification metric and is equivalent to $-1 * \log$ (likelihood function) where the likelihood function suggests how likely the model

thinks the observed set of outcomes was. Since the likelihood function provides very small values, a better way to interpret them is by converting the values to log and the negative is added to reverse the order of the metric such that a lower loss score suggests a better model.

Gain and Lift Charts

Gain and lift charts are tools that evaluate model performance just like the confusion matrix but with a subtle, yet significant difference. The confusion matrix determines the performance of the model on the whole population or the entire test set, whereas the gain and lift charts evaluate the model on portions of the whole population. Therefore, we have a score (y-axis) for every % of the population (x-axis). Lift charts measure the improvement that a model brings in compared to random predictions. The improvement is referred to as the 'lift'.

K-S Chart

The K-S chart or Kolmogorov-Smirnov chart determines the degree of separation between two distributions – the positive class distribution and the negative class distribution. The higher the difference, the better is the model at separating the positive and negative cases.

Check Your Progress 1:

1. Define Classification in Data Mining.
.....
.....
.....
2. What are the steps involved in building a Classification Model? Explain.
.....
.....
.....
3. List the advantages, disadvantages and applications of Decision Tree Classification Technique.
.....
.....
.....
4. List the advantages, disadvantages and applications of Naive Bayes Classification Algorithm.
.....
.....
.....
5. List the advantages, disadvantages and applications of SVM Classification Algorithm.
.....
.....
.....

10.9 SUMMARY

In this unit we had classification, classification techniques and model evaluation/selection.

Classification is one of the most commonly used data mining techniques. It can be used for both categorical and numerical attributes. The goal is to predict the class labels of new, unseen observations by using training data consisting of both labeled and unlabeled examples. This method makes use of an algorithm to identify patterns in the training data that are predictive of new observations.

Classification Techniques are methods of data analysis that can be used to determine the categorization of an individual based on their personal attributes. These techniques help us better understand individuals by grouping them together depending on their lifestyle, habits, and traits.

Decision Tree: A decision tree is a class discriminator that iteratively splits the training set until each partition contains only or primarily samples from one class. A split point is a test that describes how data is partitioned in each non-leaf node of the tree based on one or more qualities.

Naive Bayes: Naive bayes is used to work with probabilistic models and majorly used in machine learning. In this model, probability is calculated for each class to determine their categorization, which is then used to forecast the values for a new class.

Rule Based Classification: “If-then” rules are the classification rules, and the rule is a condition. The rules of individuals are ranked. Rule-based order refers to the order that is based on their quality. Class-based ordering refers to the grouping of rules that belong to the same class. A good rule should be error-free and cover as many scenarios as possible.

Support Vector Machine: Support Vector Machines (SVM), is a classification technique that can be used to build both classifiers and non-parametric regression models. SVM works by finding an optimal hyperplane that separates objects of different classes in the input space based on their training samples. A new classification method for both linear and non-linear data is the Support Vector Machine. It transforms the original training data into a higher dimension using a non-linear mapping. It searches for the linear optimal separation hyper plane with the additional dimension (i.e.. “decision boundary”).

A hyper plane with a good non-linear mapping to a high enough dimension can always divide data into two groups. SVM uses support vectors (“important training tuples”) and margins (specified by the support vectors) to discover this hyper plane. SVM is used for classification as well as prediction.

More on Classification can be studied in your third semester MCS-224 Artificial Intelligence and Machine Learning course.

In the next unit, we will focus on Clustering.

10.10 SOLUTIONS/ANSWERS

Check Your Progress 1:

1.

Classification is a process that assigns an object or event to one of the predefined classes in a group. It's based on their characteristics in order to be able to predict their future behavior. Classification methods are used when the data set has already been divided into groups before the classification process begins. The accuracy often depends on the preprocessing of the data which involves data cleaning (missing values, null values and blank values), data integration from multiple sources, data transformation and discretization.

Classification is a single step in the data mining process. It is used for organizing objects based on some key features. Several approaches such as the K-Nearest Neighbors classification, Decision Tree Learning and Support Vector Machines are employed for data mining classification.

2.

Following are the steps involved in building a classification model:

- **Initialize** the classifier to be used.
- **Train the classifier:** All classifiers in scikit-learn uses a `fit(X, y)` method to fit the model(training) for the given train data X and train label y.
- **Predict the target:** Given an unlabeled observation X, the `predict(X)` returns the predicted label y.
- **Evaluate** the classifier model

3.

Advantages of Decision Tree Classification Algorithm

- This algorithm allows for an uncomplicated representation of data. So, it is easier to interpret and explain it to executives.
- Decision Trees mimic the way humans make decisions in everyday life.
- They smoothly handle qualitative target variables.
- They handle non-linear data effectively.

Disadvantages of Decision Tree Classification Algorithm

- They may create complex trees which sometimes become irrelevant.
- They do not have the same level of prediction accuracy as compared to other algorithms.

Applications of Decision Tree Classification Algorithm

- **Sentiment Analysis:** It is used as a classification algorithm in text mining to determine a customer's sentiment towards a product.
- **Product Selection:** Companies can use decision trees to realize which product will give them higher profits on launching.

4.

Advantages of Naive Bayes Classification Algorithm

- It is simple, and its implementation is straightforward.
- The time required by the machine to learn the pattern using this classifier is less.
- It performs well in the case where the input variables have categorical values.
- It gives good results for complex real-world problems.
- It performs well in the case of multi-class classification.

Disadvantages of Naive Bayes Classification Algorithm

- It assumes independence among feature variables which may not always be the case.
- We often refer to it as a bad estimator, and hence the probabilities are not always of great significance.
- If, during the training time, the model was not aware of any of the categorical variables and that variable is passed during testing, the model assigns 0 (zero) likelihood and thus substitutes zero probability referred to as 'zero frequency.' One can avoid this situation by using smoothing procedures such as Laplace estimation.

Applications of Naive Bayes Classification Algorithm

- **Spam Classification:** Identifying whether an e-mail is a spam or not based on the content of the e-mail
- **Live Prediction System:** This model is relatively fast and thus predicts the target variable in real-time.
- **Sentiment Analysis:** Recognising feedback on a product and classifying it as “positive” or “negative”.
- **Multi-Class Prediction:** Naive Bayes works well for multi-class classification machine learning problems.

5.

Advantages of SVM Classification Algorithm

- It makes training the dataset easy.
- It performs well when the data is high-dimensional.

Disadvantages of SVM Classification Algorithm

- It doesn't perform well when the data has noisy elements.
- It is sensitive to kernel functions, so they have to be chosen wisely.

Applications of SVM Classification Algorithm

- **Face Detection:** It is used to read through images (an array of pixel numbers) and identify whether it contains a face or not based on usual human features.

- **Image Classification:** SVM is one of the image classification algorithms used to classify images based on their characteristics.
- **Handwritten Character Recognition:** We can use it to identify handwritten characters.

10.11 FURTHER READINGS

1. Data Mining: Concepts and Techniques, 3rd Edition, Jiawei Han, Micheline Kamber, Jian Pei, Elsevier, 2012.
2. Data Mining, Charu C. Aggarwal, Springer, 2015.
3. Data Mining and Data Warehousing – Principles and Practical Techniques, Parteek Bhatia, Cambridge University Press, 2019.
4. Introduction to Data Mining, Pang Ning Tan, Michael Steinbach, Anuj Karpatne, Vipin Kumar, Pearson, 2018.
5. Data Mining Techniques and Applications: An Introduction, Hongbo Du, Cengage Learning, 2013.
6. Data Mining : Vikram Pudi and P. Radha Krishna, Oxford, 2009.
7. Data Mining and Analysis – Fundamental Concepts and Algorithms; Mohammed J. Zaki, Wagner Meira, Jr, Oxford, 2014.